AGROCHAIN

A

WEB3 REGISTRY

PROJECT SYNOPSIS

AGROCHAIN

An Anchored Web3 Transparency Registry and Peer-to-Peer Micro-Loan Escrow Infrastructure for Agricultural Supply Chains

Submitted in partial fulfillment of the requirements for the B.Tech Degree in Computer Science & Engineering

Submitted By:

Student Name: Rajesh Patel
USN / Roll: 411001-CSE
Batch: 2022 - 2026

Under the Guidance of:

Guide Name: Dr. Anita Sharma
Designation: Professor, Dept. of CSE
Accreditation: NABL-9876 Tech Advisor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

AGROCHAIN TRANSPARENCY LABS & PARTNER COLLEGE

Academic Year: 2025 - 2026

Certificate of Approval

This is to certify that the engineering project synopsis entitled **"Design and Implementation of AgroChain: An Anchored Web3 Transparency Registry and Peer-to-Peer Micro-Loan Escrow Infrastructure for Agricultural Supply Chains"** is a genuine record of work carried out by **Rajesh Patel** bearing Roll No: **411001-CSE** in partial fulfillment for the award of B.Tech degree in Computer Science & Engineering. The work presented in this synopsis has been assessed and approved for academic submission.

Project Guide / Supervisor
Dr. Anita Sharma
Professor, CSE Dept.

Head of Department (HOD)
Dr. S. K. Nair
Professor & HOD, CSE Dept.

Declaration of Originality

I, **Rajesh Patel**, hereby declare that this project synopsis is my own original work conducted under the supervision of Dr. Anita Sharma, Department of Computer Science & Engineering. All assistance received in preparing this synopsis, including source code modifications, deployment scripts, database caching, and references, has been fully acknowledged. This work has not been submitted previously to any other university or institution for any degree.

Date: July 5, 2026
Place: Thrissur, Kerala

Student Signature
Rajesh Patel
USN: 411001-CSE

Acknowledgements

I express my deepest gratitude to my project guide, Dr. Anita Sharma, for her critical guidance, and to the department staff who facilitated our local block explorer testing and Neon Postgres cloud configurations. I also thank our agricultural partners who provided local Kerala administrative jurisdiction details to test the geographical routing priority hierarchy.

Table of Contents

System Abstract

Today's agricultural supply chain faces a double-sided trust problem. Consumers have no reliable way to verify organic labels, food origin, or certifications, making them vulnerable to fraudulent labeling. Meanwhile, smallholder farmers are shut out of traditional banking systems due to rigid collateral rules, forcing them to turn to high-interest predatory money lenders. This project presents **AgroChain**, a hybrid Web2/Web3 platform that secures agricultural traceability and offers peer-to-peer (P2P) micro-loans. By anchoring key milestones (crop lifecycles, inspector verifications, and lab test certifications) on a public Ethereum ledger, AgroChain guarantees tamper-proof records. We implement a local Flask SQL cache to prevent slow RPC calls, and a responsive React interface with MetaMask. Key contributions include: (1) geographical verifier allocation using Kerala's administrative hierarchy, (2) cryptographic signature wallet-linking, (3) strict verification guardrails that block labs from certifying unverified crops, and (4) a public explorer with built-in QR scanning.

CHAPTER 1: INTRODUCTION

1.1 Research Context

Modern agricultural supply chains have become highly complex global networks. This complexity creates a trust gap between the producer and the final consumer. Centralized supply chain databases can easily be modified or deleted by administrators, making them unreliable for verifying labels such as "Organic" or "Fair-Trade." Utilizing decentralized public ledgers (blockchains) allows for permanent, tamper-proof logs that restore trust to consumers.

1.2 System-level Problem Statement

The project addresses four core systemic failures in agricultural supply chain systems: (1) Labeling fraud due to the modify-permissions of database administrators; (2) High-interest predatory lending systems affecting small farmers lacking bank collateral; (3) Excessive retail middlemen who inflate consumer prices while shrinking farmer profit margins; (4) Scattered records where soil quality, inspections, and grades are isolated in disparate silos.

1.3 Project Engineering Objectives

The primary engineering objectives of AgroChain are to: (1) Deploy Solidity contracts on Ethereum to track crops immutably; (2) Build a location-routing algorithm that assigns local inspectors using Kerala's administrative hierarchies; (3) Authenticate inspectors using MetaMask personal message signatures; (4) Launch a P2P micro-loan marketplace where investors fund farmers directly using smart-contract escrows; (5) Build a public block explorer page with a web-camera QR scanner.

1.4 Existing Methodologies vs. AgroChain Approach

Operational Vector	Standard Database (Web2)	AgroChain Decentralized System
Data Immutability	Database records can be modified by system administrators.	Immutable; secured by cryptographic blocks on the Ethereum network.
Escrow & Funding	Relies on commercial bank approvals or cash networks.	Peer-to-peer proposal system with direct investment tracking.
Certification	Paper-based certifications that are easily duplicated.	Smart contracts require inspector verification before lab testing can occur.
Consumer Access	Consumers cannot access internal logistics databases.	Scannable packaging QR codes query the public explorer directly.
User Signatures	System actions are logged using basic database username entries.	Verifier approvals are signed using MetaMask private keys.

CHAPTER 2: LITERATURE REVIEW

2.1 Evaluation of IBM Food Trust

IBM Food Trust represents an enterprise-level traceability protocol built on Hyperledger Fabric. While highly effective at scaling tracking for corporate networks, its private permissioned architecture requires heavy licensing fees and complex backend integration. It contains no mechanism for peer-to-peer micro-financing, leaving small-scale independent farmers with no financial benefits.

2.2 Evaluation of TE-FOOD

TE-FOOD is a hybrid supply chain system designed for emerging markets, using its ONS utility token for business-to-business logging. Although TE-FOOD serves livestock traceability, it is structured around large-scale logistics operators and lacks integrated micro-loan escrows or decentralized public consumer reviews.

2.3 Evaluation of AgriDigital

AgriDigital is focused on grain supply chain traceability within the Australian market, allowing buyers and brokers to verify transactions. However, it operates as a proprietary SaaS platform, preventing standard consumers from open verification of data layers and missing integrated public quality-rating systems.

CHAPTER 3: REQUIREMENTS ENGINEERING & MODELING

3.1 Functional Requirements Architecture

- **FR-1: Farmer Crop Registration**: Farmers must be able to log in, specify crop attributes (crop type, expected yield, land survey details, GPS coordinates), and upload evidence photos and land records separately.
- **FR-2: Admin Inspector Creation**: Administrators can register new agricultural inspectors by entering details and generating a temporary password. Direct self-signup for inspectors is disabled to enforce trust.
- **FR-3: Inspector Verification Flow**: Inspectors must change their password on first login, connect their MetaMask wallet, and sign a cryptographic signature request (personal_sign) to activate their account.
- **FR-4: Geographic Routing Model**: Crops are assigned to active inspectors automatically based on proximity in Kerala: Priority 1 (same Taluk/Sub-District), Priority 2 (same District), and Priority 3 (district-level fallback).
- **FR-5: Inspection Approval Flow**: Inspectors can save local notes and select audit methods (PHYSICAL_VISIT, PHOTO_REVIEW, HYBRID) to the database, or sign the final verification on-chain using MetaMask.
- **FR-6: Lab Tester Onboarding**: Quality labs self-register with license details, government registrations, and certificates. Their accounts are locked as PENDING_APPROVAL until approved by the Administrator.
- **FR-7: Lab Test Certification**: Active labs receive crops harvested in their district. Testers conduct tests (moisture, purity, heavy metals) and certify grades and pricing on-chain via ProductRegistry using MetaMask.
- **FR-8: P2P Investor Escrows**: Investors browse verified crops, submit letters of intent (LOI), and lock and transfer ETH funding directly to the farmer using the MicroFinance.sol escrow contract.
- **FR-9: Explorer & QR Scanner**: A public registry explorer allows anyone to trace lot histories and scan physical packaging barcodes using a browser-based QR camera scanner (html5-qrcode).
- **FR-10: Admin Auditing Dashboard**: Admins monitor system audit logs, review and approve self-registered lab accounts, and monitor general platform stats (active user counts, fraud alerts).

3.2 Non-Functional System Guarantees

- **Security**: Access control is governed by Flask JWT auth on the API layer and OpenZeppelin Access Control on the smart contracts, restricting actions according to stakeholder roles.
- **Data Immutability**: Inspection reports, lab certifications, and micro-loan escrows are stored on-chain, preventing administrators from modifying historical records.
- **Performance Cache**: A Flask SQLAlchemy local database caches Solidity events and transaction logs, reducing slow Web3 RPC network requests and keeping UI load times under 1.5 seconds.

3.3 Software Stack Details

System Layer	Tech Selection	Role in AgroChain Architecture
Client Interface	React.js (Vite) & Tailwind CSS	Renders stakeholder dashboards, theme toggles, and scans QR codes.
Web3 Provider	MetaMask Wallet Extension	Cryptographically signs transactions and coordinates Ethereum gas.
Backend API Layer	Flask (Python 3.9+)	Manages JWT auth, Dual SMS/Email OTP, and coordinates database states.
Database Cache	SQLAlchemy ORM (SQLite/Postgres)	Stores relational profiles, local audit trail logs, and caches ledger transactions.
Ledger Engine	Solidity (v0.8.20) & Hardhat	Deploys FarmerRegistry, ProductRegistry, MicroFinance, and RatingSystem.
PDF Compilation	html2pdf.js / ReportLab	Compiles batch certificates, QR labels, and synopses.

3.4 Hardware Platform Requirements

For development and staging environments, the hardware requirements include: Intel Core i5 / AMD Ryzen 5 processor, 8 GB RAM (16 GB recommended to support simultaneous Hardhat blockchain nodes, Flask API, and Vite environments), and 500 MB of local disk space.

CHAPTER 4: ARCHITECTURAL DESIGN & DIAGRAMS

4.1 Multi-tier Hybrid Infrastructure

AgroChain splits operational tasks into a three-tier Web3 hybrid structure. The Client Tier (React/Tailwind) communicates with the MetaMask provider to process write operations on the blockchain, while calling the Application Tier (Flask API) for Web2-based logins and profile cache. The Data/Ledger Tier houses the relational cache database and the local Hardhat Ethereum node, allowing the application to handle both fast queries and immutable audit trails.

4.2 UML Diagram Specifications

Use Case Workflows

Farmer Use Cases: Register crop specifications, upload evidence, accept investment proposals, and update harvest timelines.
Inspector Use Cases: Verify land registry deeds, audit GPS coordinates, save local assessment notes, and approve crops on-chain.
Quality Lab Use Cases: Register NABL credentials, test crop moisture/purity, and issue batch certificates.
Investor Use Cases: Browse verified crop listings, submit proposals (LOI), and lock and transfer escrow funding.

Crop Lifecycle State Machine

Crops progress through the following sequential states:
1. **PENDING:** Farmer registers the cultivation details. Inspector is auto-assigned.
2. **VERIFIED:** Inspector conducts the audit and signs the verification on-chain.
3. **HARVEST_COMPLETED:** Farmer updates status. Crop enters the Lab testing queue.
4. **PRODUCT_AVAILABLE:** Quality lab certifies testing grades and price, publishing the lot.
5. **FUNDING_COMPLETED:** Investor locks and releases escrow funds. Crop enters the supply market.

Sequence of Transactions

1. Farmer registers crop details. Flask API stores details in DB as PENDING.
2. Inspector reviews the registration, connects MetaMask, and triggers the `approveFarmer` transaction on the Solidity contract.
3. The smart contract emits a `FarmerApproved` event. Flask API intercepts and updates status to VERIFIED.
4. Farmer harvests and updates timeline. Tester runs checks, calls `registerProduct` on the smart contract, emitting `ProductRegistered` event.
5. Investor reviews lot, submits proposal. Farmer accepts, transferring escrow ETH on-chain.

CHAPTER 5: DETAILED IMPLEMENTATION MECHANICS

5.1 Frontend Design and UX

The React interface uses a role-based structure to show stakeholders exactly what they need. A single dashboard route checks the logged-in user's role and renders relevant cards (e.g., pending crop lists for inspectors, LOI offers for farmers, or approval queues for labs). The document center uses print-specific CSS rules that force a clean light-mode layout and hide navigation bars, sidebar panels, and backgrounds. This makes it easy for farmers to print neat packaging certificates and labels.

5.2 API Layer & Custom Decorators

The Flask API coordinates logins, SMS/Email OTP generation, and database caching. We use custom decorators to protect sensitive endpoints and verify user roles before executing requests, ensuring that inspectors cannot access admin controls, and unapproved testing labs cannot access the testing queue.

Example Python decorator implementation for Role-Based Access Control:

```
def roles_allowed(*roles):
    def decorator(f):
        @wraps(f)
        def decorated(current_user, *args, **kwargs):
            if current_user.role not in roles:
                return jsonify({'message': 'Access denied'}), 403
            return f(current_user, *args, **kwargs)
        return decorated
    return decorator
```

5.3 Smart Contract Mechanics

The platform relies on four Solidity smart contracts compiled using Hardhat:
1. **FarmerRegistry.sol:** Tracks cultivation details and inspector approval records.
2. **ProductRegistry.sol:** Generates product lot numbers, records grade certifications, and verifies on-chain that the crop was approved by a verifier before certifying.
3. **MicroFinance.sol:** Locks funding deposits in escrow, tracks interest rates, and handles P2P loan payouts.
4. **RatingSystem.sol:** Stores consumer trust reviews, locking rating coordinates to prevent manipulation.

CHAPTER 6: SYSTEM TESTING & VERIFICATIONS

6.1 Testing Methodology

We tested AgroChain in three stages. First, we ran unit tests on the Solidity contracts using Hardhat and Chai, and tested Flask routes using PyTest. Second, we performed integration testing to check that blockchain event listeners correctly update the SQLite database. Finally, we completed end-to-end user testing to verify crop creation, location assignment, lab testing, P2P proposal acceptance, and explorer QR scanning.

6.2 Verification Matrix

Test Scenario	Inputs	Expected Output	Result
Crop Registration	Yield: 5000, Survey: 242/A	Crop saved as PENDING, inspector auto-assigned.	**PASSED**
Inspector Notes	Remarks: '', Approved: True	Warning modal displays: Remarks required.	**PASSED**
On-Chain Verification	MetaMask sign, Remarks: 'Ok'	On-chain TX signed, DB shifts to VERIFIED.	**PASSED**
Lab Certification Guard	Crop ID: 1 (Unverified by inspector)	Solidity transaction fails: Verification required.	**PASSED**
P2P Loan Proposal	Price: 20K, Margin: 12%, Lot: 101	Proposal saved in DB, Farmer receives badge.	**PASSED**
Accept Loan Proposal	Farmer click 'Accept'	Escrow locks, timeline shifts to FUNDING_COMPLETED.	**PASSED**
Explorer Lookup Query	Navigate to /explorer?lot=1001	Auto-loads and renders lot details & QR.	**PASSED**

CHAPTER 7: RESULTS, OUTPUTS & DISCUSSION

7.1 Interface Behavior & Printable Documents

Upon crop verification, the Farmer's dashboard displays a button to print the "Approval Letter." Once quality lab testing completes, a custom gold-bordered "Batch Quality Certificate" is issued. The CSS overrides remove buttons and sidebars to print standard A4 certificates containing a dynamic QR code. Consumers scanning the physical QR packaging label are redirected to the Explorer page showing the ledger logs.

7.2 Production Cloud Deployment

To evaluate performance under actual cloud network constraints, we successfully deployed the application to production hosting services, linking React, Flask, SQLite cache, and Postgres.
Live Web URL: https://agrochain-i6zh.onrender.com
Hosting Infrastructure: Render.com utilizing multi-stage Docker builds.
Database Engine: Neon Serverless PostgreSQL (AWS US-East).

We containerized the platform using a multi-stage Docker setup. Stage 1 compiles the React code with Vite. Stage 2 sets up the Flask backend, installs Node and Hardhat to host a local simulated blockchain node, copies static React files, and starts Gunicorn on port 5000.

7.3 Database Migrations & Sequences

Moving data from SQLite to Neon Postgres required mapping integer booleans (0 and 1) to strict Postgres booleans. We wrote `migrate_to_neon.py` to handle this transfer. After migration, new database inserts failed due to primary key sequence mismatches. To resolve this, we executed `reset_sequences.py` to synchronize Postgres sequences with the highest existing primary keys, fixing primary key collisions and restoring write functionality.

CHAPTER 8: CONCLUSION & FUTURE SCOPE

8.1 Summary of Contributions

AgroChain addresses agricultural supply chain transparency and micro-finance challenges. By integrating Ethereum smart contracts with a fast, cached web dashboard, we provide tamper-proof crop lifecycles and transparent P2P loan escrows. Features like regional inspector routing, printable QR label certificates, and cryptographic signature matching establish a secure, direct link between farmers, inspectors, quality labs, investors, and consumers.

8.2 Future Enhancement Roadmaps

AI Disease Diagnostics: Integrate computer vision to let farmers scan crop leaves and diagnose plant diseases during registration.
IoT Storage Tracking: Connect temperature and humidity sensors to log transit conditions on-chain.
Government land APIs: Link with land registry databases to auto-verify survey numbers.
Mobile Applications: Build a React Native mobile app to support offline audits in remote areas.

REFERENCES

1. S. Nakamoto, 'Bitcoin: A Peer-to-Peer Electronic Cash System,' 2008.
2. G. Wood, 'Ethereum: A Secure Decentralized Generalised Transaction Ledger,' Ethereum Yellow Paper, 2014.
3. M. Syed et al., 'Blockchain for Agricultural Supply Chain Traceability: A Review,' IEEE Access, 2021.
4. F. Tian, 'An Agri-Food Supply Chain Traceability System Based on RFID & Blockchain,' IEEE ICSSSM, 2016.
5. M. Du, Q. Chen, and Y. Xiao, 'Supply Chain Traceability System Based on Blockchain,' IEEE Access, 2020.
6. J. Lin et al., 'Blockchain and IoT integration in agriculture: Minimized trust protocols,' Comp. Electronics Agri., 2021.
7. S. Kamble, A. Gunasekaran, and H. Arimura, 'Blockchain Technology Adoption in Indian Agriculture,' Trans. Res. Part E, 2020.
8. OpenZeppelin Access Control Contracts Documentation, 2025.
9. Ethers.js v6 Documentation, Ethereum Wallet Library, 2025.

APPENDICES

Appendix A: Solidity Smart Contract Sample

The following snippet shows the on-chain certification logic from `ProductRegistry.sol`. It verifies that a farmer was approved by an inspector before the lab tester can register the crop lot.

```
function registerProduct(
    uint256 _lotNumber, uint256 _farmerId, string memory _cropName,
    string memory _qualityGrade, uint256 _price, uint256 _testDate,
    uint256 _expiryDate, string memory _certificationStatus
) public onlyRole(TESTER_ROLE) {
    require(farmerRegistry.isFarmerApproved(_farmerId),
            "Farmer registration must be approved first");
    require(!products[_lotNumber].exists,
            "Product lot number already registered");
    products[_lotNumber] = Product({
        lotNumber: _lotNumber, farmerId: _farmerId,
        cropName: _cropName, qualityGrade: _qualityGrade,
        price: _price, testDate: _testDate,
        expiryDate: _expiryDate, exists: true,
        certificationStatus: _certificationStatus,
        testerAddress: msg.sender
    });
    emit ProductRegistered(_lotNumber, _farmerId, _qualityGrade);
}
```

Appendix B: REST API Specifications Sample

Register User (`POST /api/auth/register`)
Accepts user registration profiles (FARMER, TESTER, CONSUMER, INVESTOR) along with verification code OTP signatures.
Response: `{'message': 'User registered successfully!'}`

Register Cultivation (`POST /api/farmer/register`)
Allows farmers to submit expected yields, GPS coordinates, and documents.
Response: `{'message': 'Crop registered successfully', 'id': 1}`

Appendix C: Operational User Guide

Farmer User Workflow

1. Go to `/register`, select Farmer role, verify using SMS/Email OTP.
2. Register your crop on the dashboard: specify GPS, expected yields, and upload land survey deeds.
3. Once verified by the inspector, print the official approval letter from the dashboard Crop History page.
4. Update status to HARVEST_COMPLETED when ready to move the crop into the testing queue.
5. Print the gold-bordered Quality Certificate and QR labels to stick on crop bags.

Inspector User Workflow

1. Request temporary login credentials from the system administrator (no self-registration).
2. Change temporary password on first login to unlock dashboard.
3. Link MetaMask wallet and sign cryptographic validation request to change status to ACTIVE.
4. Access assigned inspections (routed based on Kerala sub-districts).
5. Save notes locally (no gas required) or sign the final on-chain approval via MetaMask.

Quality Lab User Workflow

1. Register as a TESTER, upload lab accreditation details and licenses.
2. Wait for administrator to approve account and change status to ACTIVE.
3. Link MetaMask wallet to allow on-chain transaction certification.
4. View crops harvested in your ZIP/District, input testing grades, and approve using MetaMask.

Consumer Explorer Workflow

1. Open `/explorer` to search by lot number or cultivation ID, or scan a package QR code with your camera.
2. View the crop's history, inspector remarks, GPS coordinates, and testing grades.
3. Log in to submit ratings and reviews to build public seller ratings.